EX NO: 2	Developing a RESTful CRUD API Using Node.js, Express.js, MongoDB Atlas, and Mongoose
DATE:	

AIM:

To develop a backend API using Node.js, Express.js, MongoDB Atlas, Mongoose, GitHub, and Thunder Client for performing CRUD operations.

Step 1: Install Required Software

Install the following software:

- Node.js (LTS Version)
- Visual Studio Code
- Git
- MongoDB Atlas Account
- Thunder Client Extension in VS Code

Step 2: Create MongoDB Atlas Cluster

1. Login to MongoDB Atlas.
2. Create a Free Cluster (M0).
3. Create a Database User (e.g. Username: rajkumar, Password: Rajkumar2026).
4. Go to Security → Network Access → Add IP Address → 0.0.0.0/0.
5. Click Connect → Drivers → copy the connection string.

Step 3: Create Node.js Project

Open Command Prompt and create the project folder:

In the VS Code terminal, initialize the Node project and Git:

Step 4: Install Packages

Install
Express. npm
install express
Install
Mongoose.
npm install
mongoose Install
CORS.

npm install cors
Install Body
Parser.
npm install body-
parser Install
Nodemon.

npm install --save-dev nodemon

Step 5: Create Folder Structure

```
API
│
├── node_modules
├── models
│   └── Item.js
│
├── routes
│   └── itemRoutes.js
│
├── server.js
├── package.json
└── package-lock.json
```

Step 6: Modify package.json

Replace the scripts section with:
Replace the scripts section with
"scripts": {
"start": "nodemon server.js"

},

Step 7: Create Item Model (models/Item.js)

Create

models → Item.js
item.js
const mongoose = require('mongoose');

```
const ItemSchema = new mongoose.Schema({
   name: {
      type: String,
      required: true
   },

   quantity: {
      type:
      Number,
      required:
      true
   }

});

module.exports = mongoose.model('Item', ItemSchema);
```

Step 8: Create Routes (routes/itemRoutes.js)

Create

routes → itemRoutes.js

```
const express =

require('express'); const router

= express.Router();

const Item = require('../models/Item');
router.post('/', async (req, res) => {
   try {

      const newItem = new
      Item(req.body); await
      newItem.save();
```

```
        res.status(201).json(newItem);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});
router.get('/', async (req, res) =>
    { try {
        const items = await Item.find();
        res.json(items);
    } catch (err) {
        res.status(500).json({ message: err.message });
    }
});
router.put('/:id', async (req, res) =>
    { try {
        const updatedItem = await Item.findByIdAndUpdate(
            req.params.id,
            req.body,
            { new: true }
        );
        res.json(updatedItem);
    } catch (err) {
        res.status(400).json({ message: err.message });
    }
});
router.delete('/:id', async (req, res) =>
```

```
        { try {

            await
            Item.findByIdAndDelete(req.params.id);
            res.json({ message: "Item deleted" });
        } catch (err) {

            res.status(500).json({ message: err.message });

    }

});
  ;

module.exports = router;
```

Step 9: Create Server (server.js)

```
    const express = require('express');
    const mongoose =
    require('mongoose');
    const bodyParser = require('body-

    parser'); const cors = require('cors');

    const app = express();
    app.use(cors());

    app.use(bodyParser.json()

    ); mongoose.connect(

    "mongodb+srv://rajkumar:Rajkumar2026@cluster0.yekjugz.mongodb.net/labdb?retryWrites=tru
    e&w=majority&appName=Cluster0"

    )

    .then(() => console.log("MongoDB Atlas Connected"))

    .catch(err => console.log(err));
    const itemRoutes =
    require('./routes/itemRoutes');
```

```
app.use('/api/items', itemRoutes);
app.listen(5000, () => {
   console.log("Server running on port 5000");

});
```

Step 10: Start Server

Run
npm
start
Outp
ut
Server running on port
5000 MongoDB Atlas
Connected

Step 11: Test API using Thunder Client

POST

Method: POST URL: http://localhost:5000/devops2/items
Body:

```
{
  "name": "Book",
  "quantity": 10
}
```

Response:

```
{
  "_id": "6a57278f4a3de59dfccd8939",
  "name": "Book",
  "quantity": 10
}
```

GET

Method: GET URL: http://localhost:5000/ devops2/items

Response:

```
[
  {
    "_id": "6a57278f4a3de59dfccd8939",
    "name": "Book",
    "quantity": 10
  }
]
```

PUT

Method: PUT URL: http://localhost:5000/ devops2/items/6a57278f4a3de59dfccd8939

Body:

```
{
  "name": "Book",
  "quantity": 20
}
```

Response:

```
{
  "_id": "6a57278f4a3de59dfccd8939",
  "name": "Book",
  "quantity": 20
}
```

DELETE

Method: DELETE URL: http://localhost:5000/ devops2/items/6a57278f4a3de59dfccd8939

Response:

```
{
  "message": "Item deleted"
}
```

Step 12: Verify in MongoDB Atlas

Open Database → Browse Collections → labdb → items, then verify:

- POST → New document inserted.
- GET → Documents retrieved.
- PUT → Quantity updated.
- DELETE → Document deleted.

Step 13: Push Project to GitHub

```
git status
```

```
git add .
git commit -m "Completed Backend API CRUD"
git branch -M main
git remote add origin https://github.com/<username>/<repository>.git
git push -u origin main
```

OUTPUT:

VERIFY node.js AND GIT INSTALLATION:

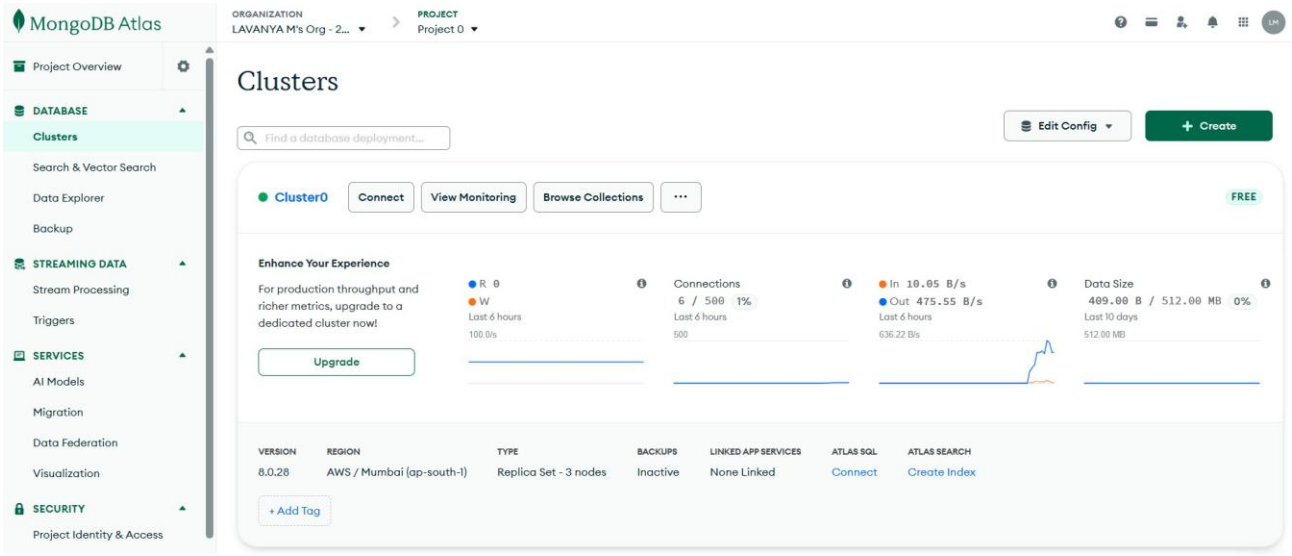
```
C:\Users\ASUS>node -v
v24.16.0

C:\Users\ASUS>npm -v
11.13.0

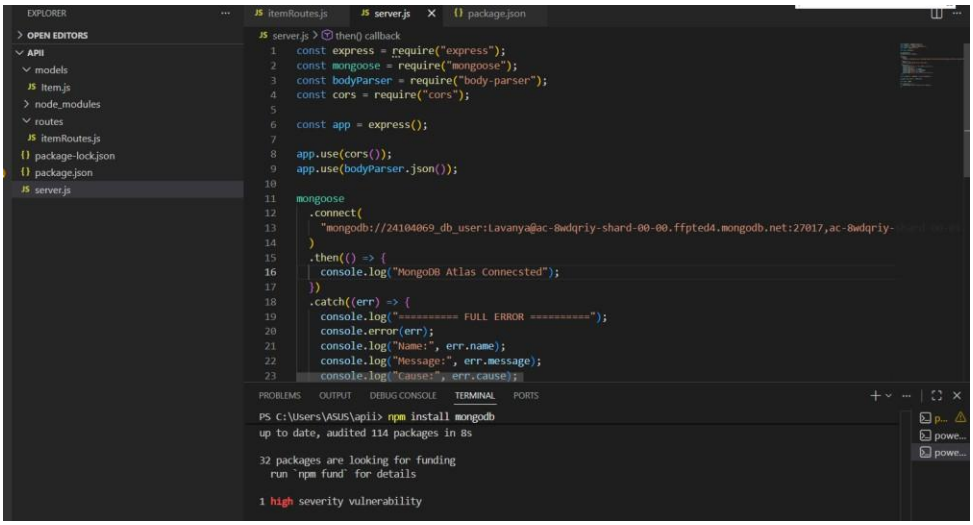
C:\Users\ASUS>git --version
git version 2.54.0.windows.1

C:\Users\ASUS>
```

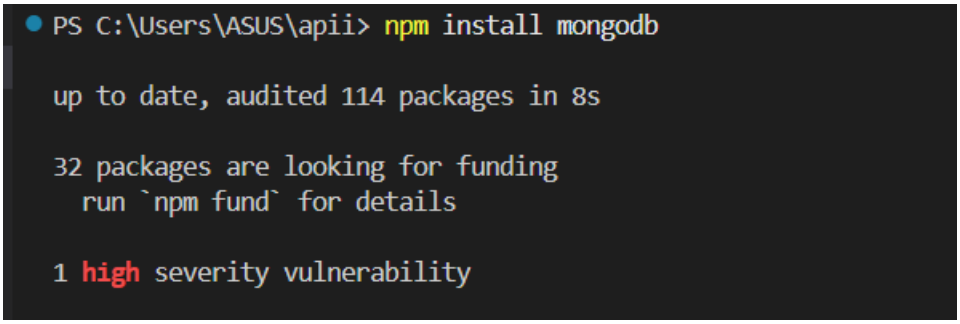
MONGODB ATLAS CLUSTER:



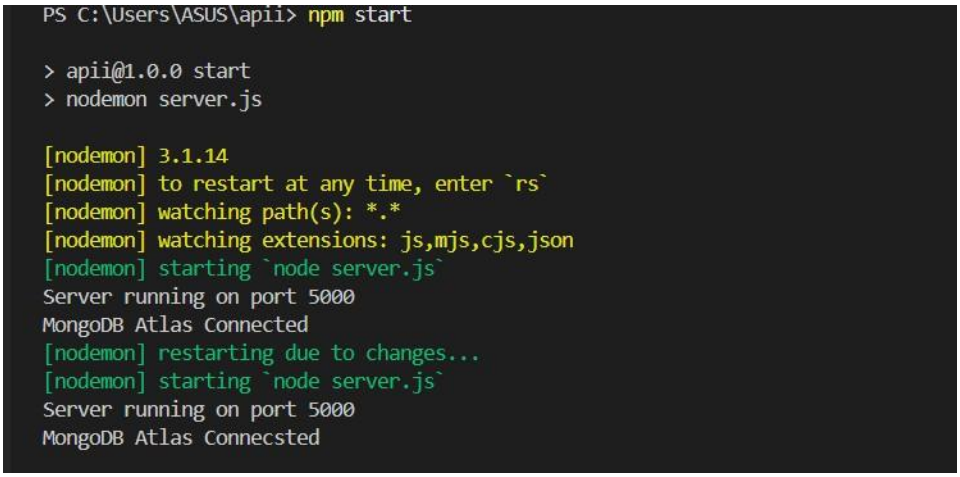
PROJECT FOLDER IN VS CODE:



INSTALLED PACKAGES:

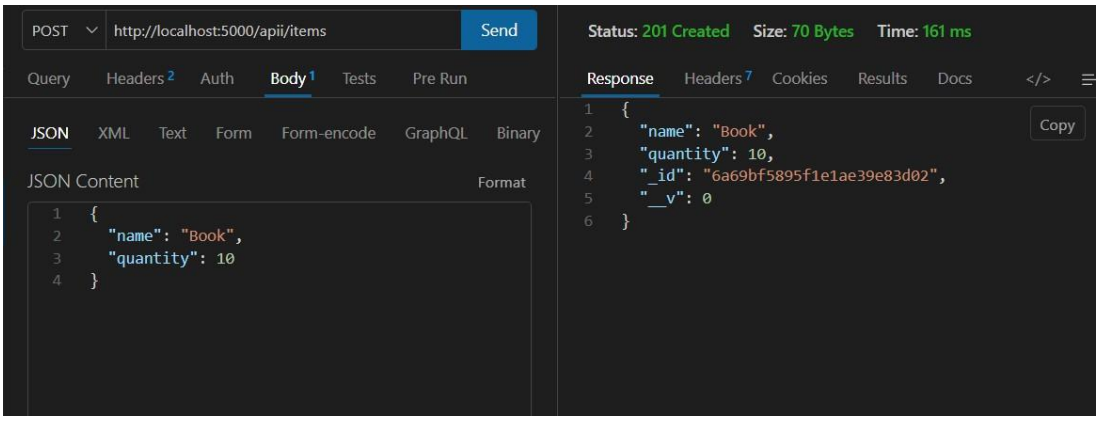


RUNNING SERVER:

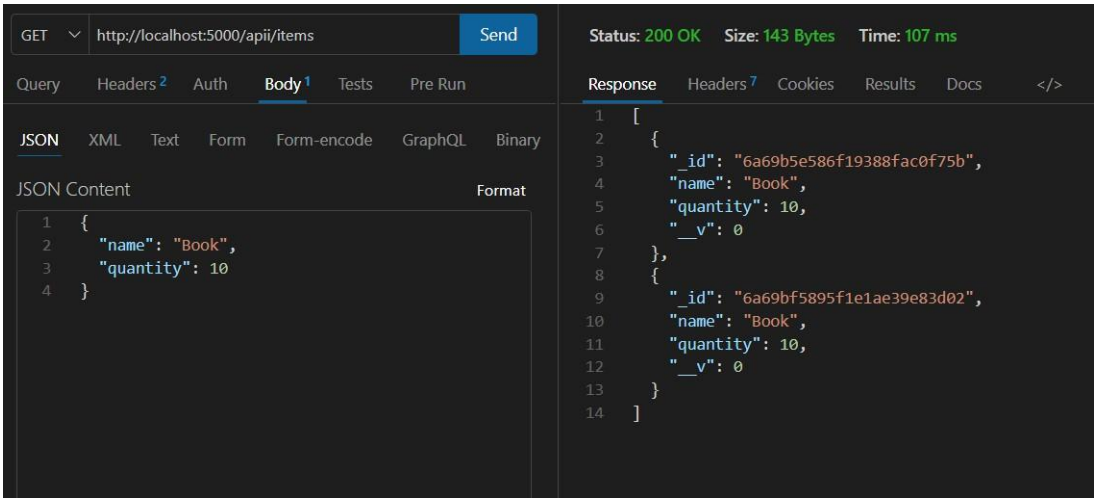


REQUEST COMMANDS:

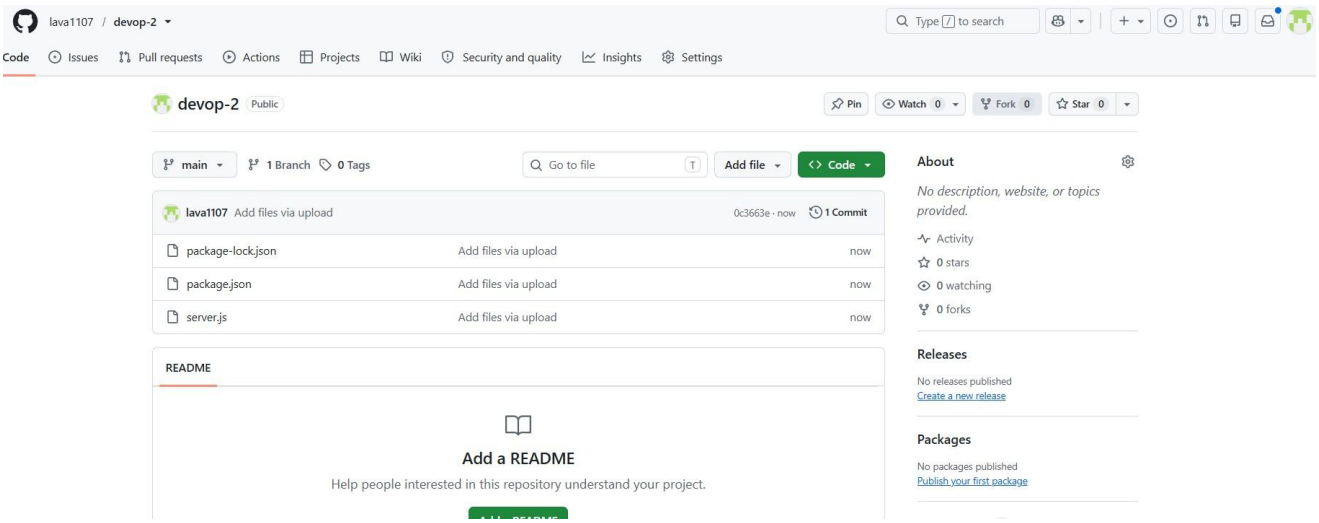
POST



GET



GITHUB PUSH:



RUBRICS

GitHub Commands Syntax & Description	Implementation & Execution	Tools / Technology Usage	Code Quality & Documentation	Time Management	Viva	Total
10	15	10	5	5	5	50

RESULT:

The RESTful CRUD API was successfully developed and tested, with all CRUD operations working correctly using MongoDB Atlas